

Concept

The previous project focused on authenticated data submitted to the Log Analytics Data Ingestion API 2.0 using a service principal. While that was fairly secure, it had a weakness in that the private keys deployed to all lab devices were identical. This can be addressed by deploying device certificates using SCEP. This certificate can be used to encrypt the connection between device and data submission endpoint via Mutual TLS (mTLS).

This allows us to move the authentication flow to the cloud, no longer requiring the JWT assertion to obtain an access token, reducing complexity of the inventory scripts.

mTLS only requires a certificate, all validation that the correct certificate is being used must be done in the Azure function. The requires a PEM of each certificate in the trust chain (Intermediate and Root CAs) be stored somewhere, and the most secure method is an Azure Key Vault.

This lab is to simulate and document this environment.

Certificates

SCEP generates subordinate certificates signed by a certificate authority (CA), in our case the extended key usage (EKU) properties of the subordinate must enable the private key to be used for the “ssl-client” purpose. We also have to simulate the trust chain, so we need a Root CA and any Intermediate CA’s public keys that can be used to verify properly signed these subordinate certificates.

The subordinate certificate CN is populated using a template in SCEP, so the best way for our inventory scripts to detect which certificate to use is a naming convention like “CN={{DeviceId}}, OU=<Identifier>” and then search all certificates to find the matching Identifier in the subject.

Get-Cert.ps1

To make the process of creating certificate files in the needed format, the Get-Cert Powershell script can automate the process in Windows for a lab without SCEP.

All the configuration variables are in the first block of code.

Update these with your values and run the script in the PowerShell_ISE, you do not need to be an administrator.

<RootFriendlyName> - The full name for the root certificate. (Ie. "Lab_Root_CA2")

<FriendlyName> - The full name for your certificate. (Ie. "AF_Collect")

<MyPassword> - The password required to access the private keys for both certificates.

<DNSName> - What you use for a domain name.

Make sure you have a record of the password, friendly name and certificate thumbprint for each certificate.

The certificates will be in your

"Documents" folder. Make sure you save these files somewhere secure.

```
Get-Cert.ps1 X
1  # -----
2  #  PARAMETERS FOR SELF-SIGNED CERT
3  #  -----
4  $RootFriendlyName = "<Root Cert Name>"
5  $FriendlyName = "<Leaf Cert Name>"
6  $MyPassword = "<Your Password>"
7  $DNSName = "<Your domain>"
8  #
9  cls
10 Write-Host "Certificate Generation begin."
```

```
Certificate Generation begin.
Root CA Thumbprint: ac463065732e5f0017b425cffce05a99e01a4308
Root CA Friendly Name: IntuneLab_ROOT_CA
Root CA Subject: CN=IntuneLab_ROOT_CA

Certificate Thumbprint: 18b6efa15ccf580fdf497619328d4a9c259851a5
Friendly Name: Data Collect
Subject: CN=Data_Collect
Certificate Generation complete.
```

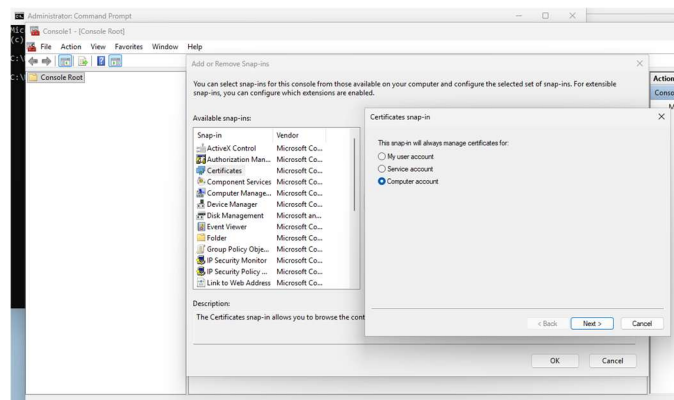
Installing the certificates manually

Windows Certificate

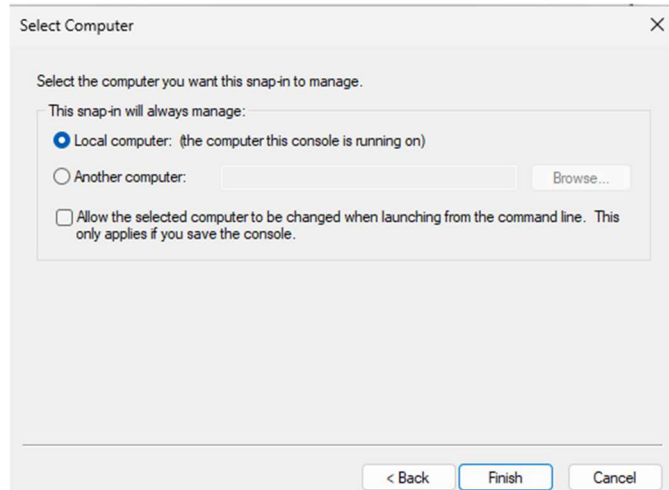
Since this is a test machine, you will be running the script as a user. The LocalMachine\My certificate store restricts users from reading the private key information from the certificate. You need to have administrative rights to perform this step.

Use the Windows search to find the icon for "CMD". Right-click on the icon and choose "Run as Administrator". Using the command prompt, run "MMC.EXE", Select "File", "Add/Remove Snapin..." and add the "Certificates" snap in.

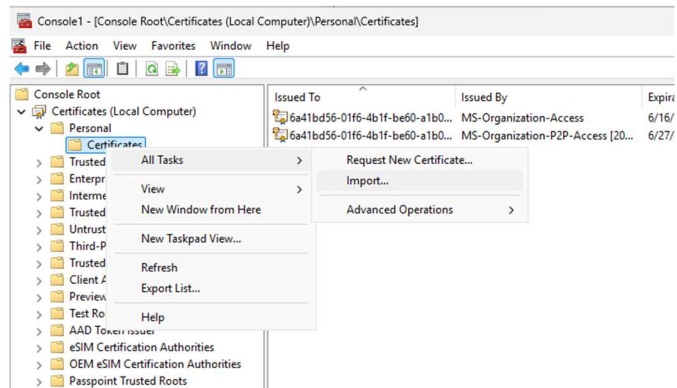
Select "Computer Account" and click "Next."



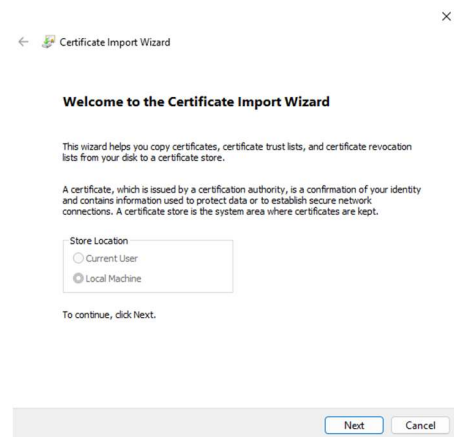
Select “Local Computer” and click “Finish”.



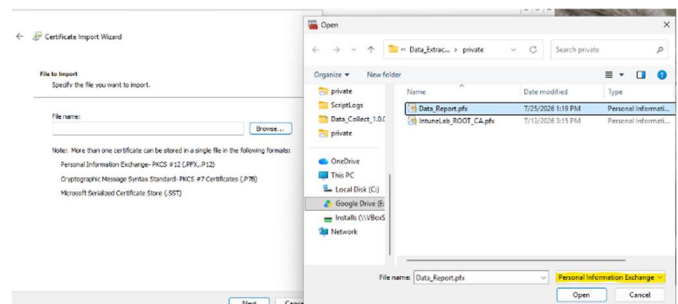
Locate the \Personal\Certificates folder in the left pane. Right-click, chose “All Tasks” and “Import...”



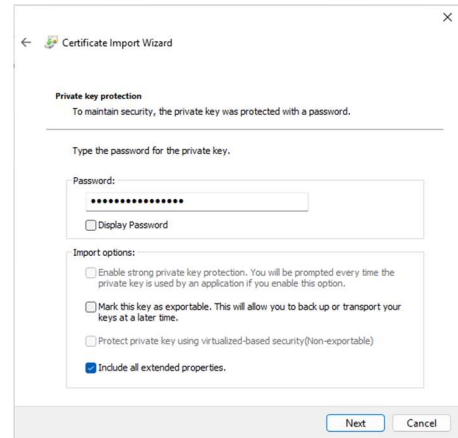
Click “Next”.



Click “Browse”, change the file type to “Personal Information Exchange”, select your .PFX file, click “open” and then click “Next”.



Enter the password and click “Next.”



Private key protection
To maintain security, the private key was protected with a password.

Type the password for the private key.

Password:

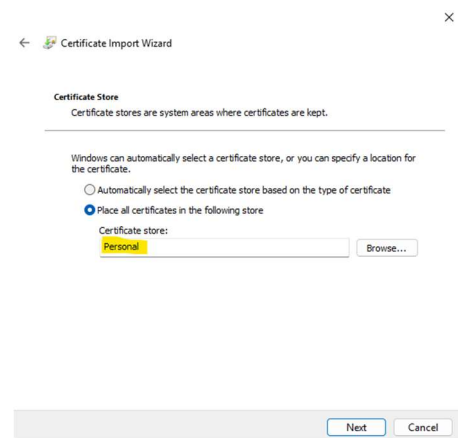
☐ Display Password

Import options:

- ☐ Enable strong private key protection. You will be prompted every time the private key is used by an application if you enable this option.
- ☐ Mark this key as exportable. This will allow you to back up or transport your keys at a later time.
- ☐ Protect private key using virtualized-based security (non-exportable)
- ☒ Include all extended properties.

Next Cancel

Make sure you have selected the “Personal” store and click “Next”.



Certificate Store
Certificate stores are system areas where certificates are kept.

Windows can automatically select a certificate store, or you can specify a location for the certificate.

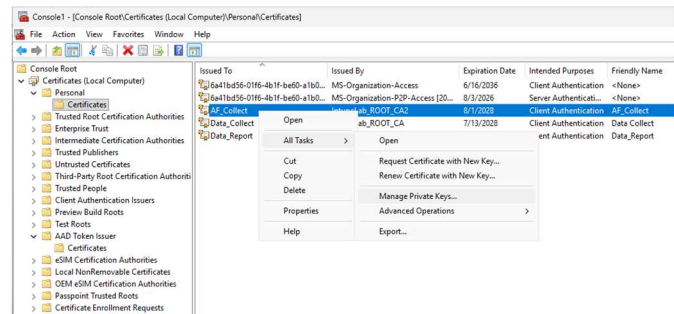
☐ Automatically select the certificate store based on the type of certificate

☒ Place all certificates in the following store

Certificate store: Browse...

Next Cancel

Right-click on the certificate, choose “All Tasks” and “Manage Private Keys...”.



Console1 - [Console Root\Certificates (Local Computer)\Personal\Certificates]

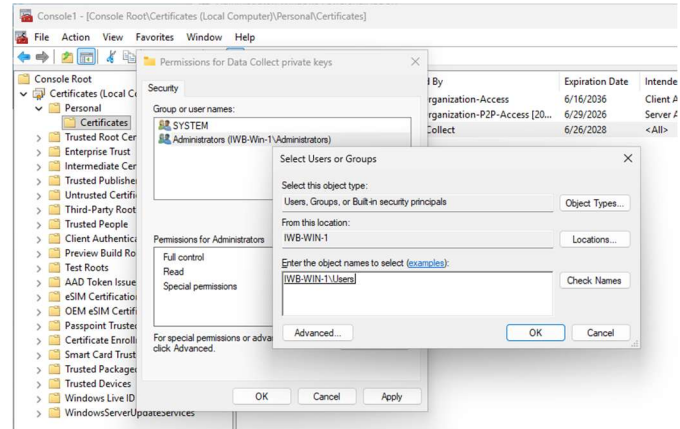
	Issued To	Issued By	Expiration Date	Intended Purposes	Friendly Name
	ba1bd56-01f6-4b1f-be60-a1b0...	MS-Organization-Access	6/16/2036	Client Authentication	<None>
	ba1bd56-01f6-4b1f-be60-a1b0...	MS-Organization-Access [20...	8/3/2036	Server Authentication	<None>
AF_Collect	ab_ROOT_CA	ab_ROOT_CA	9/1/2028	Client Authentication	AF_Collect
Data_Collect				Client Authentication	Data_Collect
Data_Report				Client Authentication	Data_Report

Context menu options: Open, All Tasks, Cut, Copy, Delete, Properties, Help, Request Certificate with New Key..., Renew Certificate with New Key..., Manage Private Keys..., Advanced Operations, Export...

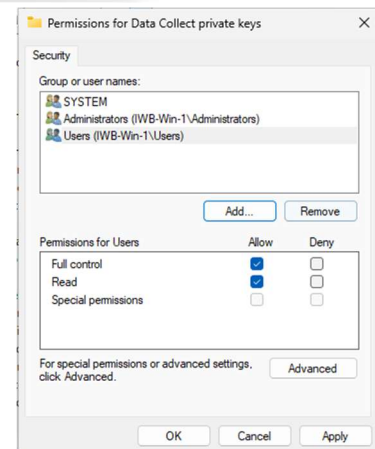
A permissions dialog will pop up. Click “Add”.

Enter “Users” in the bottom box, click “Check Names” to ensure the group or account can be resolved.

Click “OK”.



Verify the group or account has at least “Read” and “Full Control” access. You need at least “Read”, but “Full Control” will be needed to delete the certificate later.



MacOS Certificates

The certificate installation on MacOS is easier to do using Terminal and a few commands. Copy the PFX file for the leaf certificate into your documents folder.

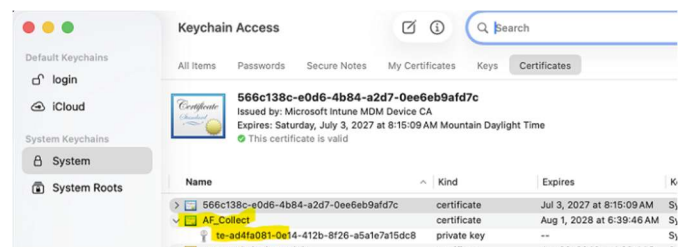
Start the Terminal app and navigate to your documents folder.

Run the following commands:

```
sudo security add-trusted-cert -d -r trustRoot -k/Library/Keychains/System.keychain <Root CA Cert .cer file name>
```

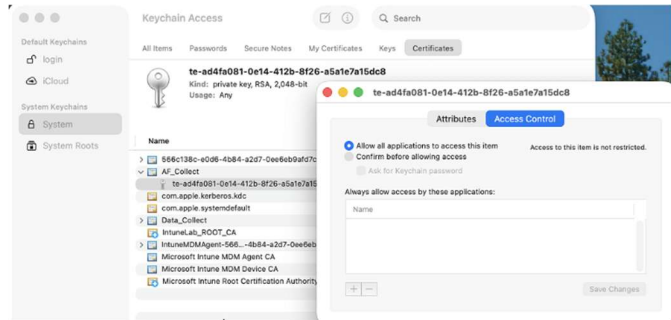
```
sudo security import <Leaf Cert .pfx file name> -k
```

```
/Library/Keychains/System.keychain -P  
"<PFX file password>" -A
```



Using the Keychain Access app, you can now confirm:

- The Leaf certificate is installed with its private key visible.
- “Allow all applications to access this item” is set on the private key.



You can verify the certificate is properly installed if it is listed in the output from “security find-identity -v -p ssl-client”.

Linux Certificates

<This section Pending>

Alter and test the scripts

This is the stage where you modify the Inventory for your own purposes.

Alter the Windows Inventory Script and test

We are going to use a Windows PowerShell script for testing (Windows_Custom_Inventory.ps1). You will need to run the script using administrative access.

Enter “Powershell_ise” in the Windows Search box. Locate the icon for “Windows PowerShell ISE 5.1”, right-click on it and choose “Run as Administrator”.

You will likely need to run “Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy Unrestricted” in the console.

Our first goal is to make sure the Inventory script produces a valid JSON file, and the certificate we need can be accessed.

The first few lines of the script are configuration variables.

Populate:

- \$CertName
- \$LogFile
-

```
1 #  
2 # BEGIN CONFIGURATION  
3 #  
4 # Modules  
5 Add-Type -AssemblyName System.Web  
6 #  
7 # Certificate (Required for mTLS)  
8 $CertName = "AF_Collect"  
9 # URL Data (for JSON submission)  
10 $URL = "<URL>"  
11 # Script logging  
12 $LogFile = "C:\ProgramData\ScriptLogs\Inventory.log"  
13 #  
14 # If a log file has been specified and it already exists, delete it.  
15 if (-not [string]::IsNullOrEmpty($LogFile) -and (Test-Path -Litera  
16     try {
```

The next block of code are the functions that provide functionality to find the correct certificate and update the log.

At the beginning of the script, we use those functions. The script will not run unless a valid certificate is found.

```
157 # BEGIN SCRIPT
158 #
159 #
160 # Get the first valid certificate that matches $CertName
161 $Cert = Get-ValidCertificate -CertName $CertName
162 # If one was found
163 if ($Cert) {
164     # Get the thumbprint
165     $Thumbprint = (Get-Childitem -Path "Cert:\LocalMachine\My" | Where-Object {$_.Subject -Match $CertName}).Thumbprint
166     # BEGIN INVENTORY
167 }
```

Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and MacIntosh, remember that your JSONs must have identical structures and plan accordingly.

```
167 # BEGIN INVENTORY
168 #
169 #
170 #
171 # Get the ManagedDeviceID and Name from Intune enrollment
172 try {
173     # If the device is enrolled
174     if (@(Get-Childitem HKLM:\SOFTWARE\Microsoft\Enrollments\ -Recurse |
175         # Get the source for the managed device information
176         #
```

Use the code example to create the structure we are going to export to JSON.

```
422 #
423 # Compile all the settings in
424 $Inventory = New-Object System.Collections.Generic.List`1[System.Object]
425 $Inventory.Add-Member -MemberType NoteProperty -Name "DefAVSipage" -Value $Defender_AVSipage -Force
426 $Inventory.Add-Member -MemberType NoteProperty -Name "DefFirmware" -Value $Defender_Firmware -Force
427 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLocker_State -Force
428 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLocker_Start -Force
429 $Inventory.Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
430 $Inventory.Add-Member -MemberType NoteProperty -Name "DefAVSipage" -Value $Defender_AVSipage -Force
431 $Inventory.Add-Member -MemberType NoteProperty -Name "DefFirmware" -Value $Defender_Firmware -Force
432 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLocker_State -Force
433 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLocker_Start -Force
434 $Inventory.Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
435 $Inventory.Add-Member -MemberType NoteProperty -Name "DefAVSipage" -Value $Defender_AVSipage -Force
436 $Inventory.Add-Member -MemberType NoteProperty -Name "DefFirmware" -Value $Defender_Firmware -Force
437 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLocker_State -Force
438 $Inventory.Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLocker_Start -Force
439 $Inventory.Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
440 #
441 # Convert to JSON for sending
442 $Body = $Inventory | ConvertTo-JSON
443 #
444 #
445 #
446 #
447 #
448 #
449 # END INVENTORY
450 #
```

```
357 # UnComment for DEBUGGING
358 $Body
359 exit 0
360 #
361 #
362 #
```

```
PS C:\WINDOWS\system32> E:\My Drive\Repo\Intune_Lab\Projects\Custom_Reporting_AF\privat
[
  {
    "ManagedDeviceID": "Lenovo_ThinkPad_X1_2018_1111 AM",
    "ManagedDeviceName": "c6e23685-79af-4d76-9469-098d32c3d3e",
    "DefenderState": "Running",
    "DefenderStart": "Automatic",
    "DefAVSipage": 0,
    "DefFirmware": "1.1.26060.3008",
    "BitLockerState": "Stopped",
    "BitLockerStart": "Manual",
    "BitEncrypted": "C:\FullyDecrypted",
    "BitProtected": "C:\Unprotected",
    "BitProtector": "C:\Unknown",
    "ProtectVersion": "1.1",
    "ProtectMeta": "1",
    "ProtectLaunch": "1",
    "ProtectBackground": "1",
    "FileVault_Status": "1",
    "FileVault_UserToken": "1",
    "FileVault_BootToken": "1",
    "LastBootTime": "2020-08-10T17:25:30",
    "LastBootTime": "2020-08-10T17:25:30",
    "MAC": "Intel(R) PRO/1000 MT Desktop Adapter [08:00:27:0C4:49:3C]",
    "MAC1": "1",
    "MAC2": "1",
    "MAC3": "1",
    "MAC4": "1",
    "MAC5": "1"
  }
]
```

After the inventory data has been converted to JSON, we have a block comments can be removed to allow debugging.

Run the script.

If you did everything correctly, you should see the JSON contents.

If not, you need to fix something either with your certificate or with your inventory code.

When you have proven the script works, you can comment out the debugging code.

Copy the JSON (only the parts between “[” and “]”, and with those characters and save it as a text file. The “[” and “]” bracket make it a JSON array, so do NOT include those. Save this data to a text file with the “.json” extension. You will need it later.

Alter the MacOS Inventory Script and test

The configuration section of the script is at the start.

Populate:

- IDENTITY
- LogFile

```
5 #-----
6 # CONFIGURATION
7 #-----
8 # A substring we use to find the private key
9 IDENTITY="AF_Collect"
10 # URL (for JSON submission)
11 URL="<URL>"
12 # LOG FILE
13 LogFile="/Library/Microsoft/CustomInventory.log"
14 #-----
15 # Logging initialization
```

Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and Macintosh, remember that your JSONs must have identical structures and plan accordingly.

Use the code example to create the structure we are going to export to JSON.

After the inventory data has been converted to JSON, we have a block where breakpoints can be set to allow debugging.

Uncomment this code.

Run the Inventory script using “sudo bash MacOS_Inventory.sh”.

If you did everything correctly, you would see the JSON contents followed by the access token you received.

If not, you need to fix something either with your certificate or with your inventory code.

Comment out the breakpoint when you have it working.

Alter the Linux Inventory Script and test

<This section Pending>

Set up Log Analytics

Schema (JSON)

In the older version of Log Analytics Data Ingestion API, your JSON could be fluid. MacOS and Windows devices typically have different disk encryption and anti-virus products and each

```
185 )
186 #####
187 # Begin Inventory script
188 #####
189 #-----
190 # Detect Intune Device Name
191 #-----
192 MANAGEDDEVICENAME=$(Hostname)
193 #
194 #-----
```

```
365 #
366 #-----
367 # Build json
368 #-----
369 jsonData=$(jq -n \
370   --arg ManagedDeviceName "$MANAGEDDEVICENAME" \
371   --arg ManagedDeviceID "$MANAGEDDEVICEID" \
372   --arg DefenderState "" \
373   --arg DefenderStart "" \
374   --arg DefSpySigAge "" \
```

```
307 #
308 #
309 # Debugging
310 echo "$jsonData"
311 exit 0
312 #
313 #-----
```

```
Documents -- zsh -- 120x33
ianbaxter@Ians-MacBook-Pro documents % sudo bash MacOS_Custom_Inventory.sh
Password:
{
  "ManagedDeviceName": "Ians-MacBook-Pro.local",
  "ManagedDeviceID": "56dc138c-e8d6-4b84-a2d7-9e6eb9afdf7c",
  "DefenderState": "",
  "DefenderStart": "",
  "DefSpySigAge": "",
  "DefNLSigAge": "",
  "DefAVSigAge": "",
  "DefAMEngine": "",
  "BitLockerState": "",
  "BitLockerStart": "",
  "BitEncrypted": "",
  "BitEncryption": "",
  "BitProtected": "",
  "BitProtector": "",
  "XProtect_Version": "5353",
  "XProtect_Meta": "5353",
  "XProtect_Launch": "enabled",
  "XProtect_Background": "enabled",
  "FileVault_Status": "Macintosh HD - Data: Not Encrypted",
  "FileVault_UserToken": "no console user",
  "FileVault_BootToken": "Escrowed",
  "BuildDate": "07/02/2026, 12:56:49 PM",
  "LastBootTime": "08/02/2026, 07:19:58 AM",
  "MAC0": "Thunderbolt Bridge [12:50:5b:0b:e6:c1]",
  "MAC1": "Wi-Fi [98:9c:4a:b7:7e:14]",
  "MAC2": "Thunderbolt 1 [12:50:5b:0b:e6:c1]",
  "MAC3": "Thunderbolt 2 [12:50:5b:0b:e6:c0]",
  "MAC4": "Thunderbolt 3 [12:50:5b:0b:e6:c5]"
}
```

platform could use differently formatted JSON files. The table structure would change in response to these differences. This is no longer the case.

The new API submits the JSON via a Data Collection EndPoint (DCE) which passes it to a transform process, the Data Collection Rule (DCR). Data is then filtered to ensure that anything passed to Log Analytics matches an existing table structure and has been submitted using the Azure Function's Managed Identity credentials. This means that MacOS and Windows clients must create identically formatted JSON output and submit it securely using mTLS to allow us to detect and prevent spoofing of compliance data.

This example focuses on disk encryption, anti-virus and physical network adapters since Intune only reports on wi-fi adapter details.

The Intune hardware inventory collects once in 7 days. You can use the Extended Inventory to reduce that to every one day for details it can collect. Custom Compliance scripts can be set to run every hour, proving a much more real-time data source for live dashboards.

If you have deployed Microsoft Defender to MacOS devices and don't have access to the Microsoft Defender for Endpoint Graph API's, you may have noticed the Intune Graph API for Defender only lists data for Windows devices. If you need to collect Defender data for Windows and MacOS, this may be your only option.

The DCE expects a JSON array of these objects, even if only one JSON item is in the array. The Table and DCR creation processes only use one JSON item. Windows devices can report blanks for MacOS-specific metrics and vice-versa. Make sure typical values are populated for all values in this JSON template, both Windows and MacOS values need to be combined.

Here is our template:

```
{
  "ManagedDeviceName": "IanBaxter_Windows_6/19/2026_3:13 AM",
  "ManagedDeviceID": "<INTUNE DEVICE ID>",
  "DefenderState": "Running",
  "DefenderStart": "Automatic",
  "DefSpySigAge": 1,
  "DefNisSigAge": 1,
  "DefAVSigAge": 1,
  "DefAMEngine": "1.1.26060.3008",
  "BitlockerState": "Running",
  "BitlockerStart": "Manual",
  "BitEncrypted": "C:FullyDecrypted",
  "BitEncryption": "C:None",
  "BitProtected": "C:Unprotected",
  "BitProtector": "C:Unknown",
  "XProtect_Version": "5346",
  "XProtect_Meta": "5346",
  "XProtect_Launch": "enabled",
  "XProtect_Background": "enabled",
  "FileVault_Status": "Macintosh HD - Data;Encrypted",
  "FileVault_UserToken": "enabled",
```

```

"FileVault_BootToken": "enabled",
"BuildDate": "2026-06-16T17:25:30",
"LastBootTime": "2026-07-08T13:50:03",
"MAC0": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC1": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC2": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC3": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC4": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C",
"MAC5": "Intel(R) PRO/1000 MT Desktop Adapter|08:00:27:C4:49:3C"
}

```

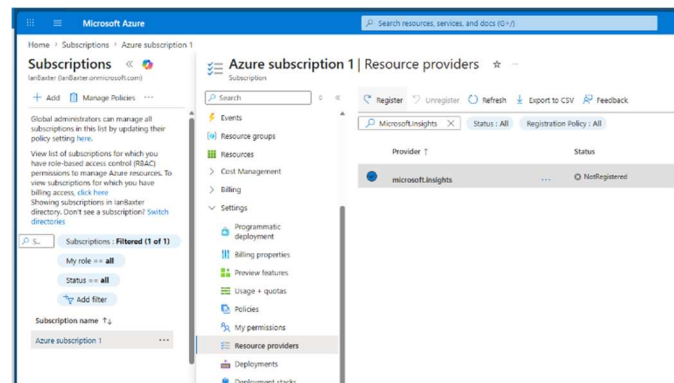
Register Microsoft Insights resource provider

In the Azure Portal, search for “Subscription” and load the applet.

Select the correct subscription. Click on “Resource Providers”, search for “Microsoft.Insights”.

Select it and click the “Register” option.

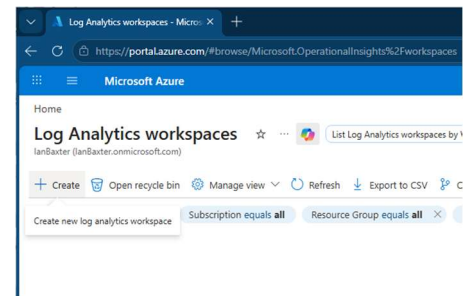
Wait for the notification that the registration has completed.



Create the Log Analytics Workspace

Launch the Azure Portal, search for Log Analytics workspaces.

Click on “Create”.

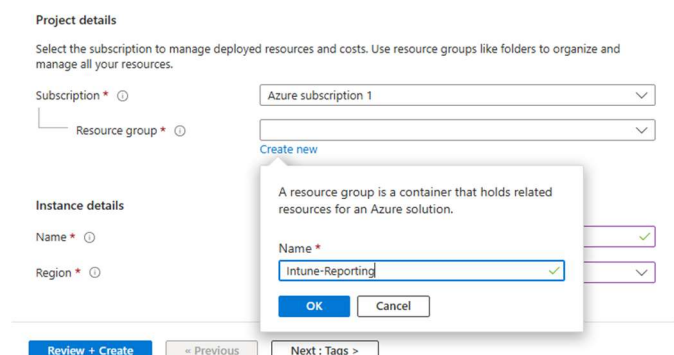


You may need to create a Resource Group. If so, click the “Create New” option.

Populate the Name and Region, click “Review + Create”.

When the validation passes, click “Create”.

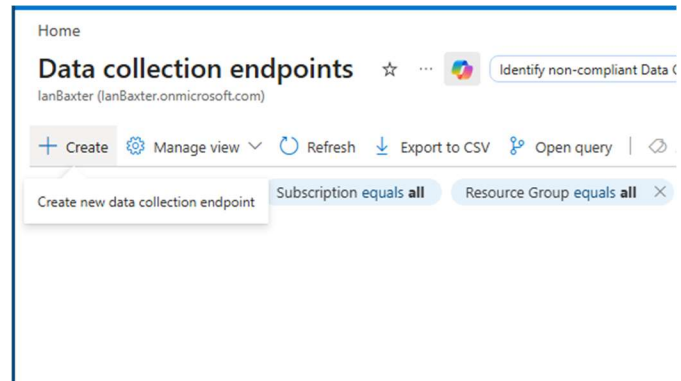
Click the “Go to Resource” option.



Create a DCE (Data Collection Endpoint)

Open the Azure Portal, search for “Data Collection Endpoints” and launch the applet.

Click the “Create” option.



Populate the “Endpoint Name”, select the appropriate Subscription, Resource Group and Region.

The Region should be the same as your Tenant and must match the values for the DCR and Log Analytics workspace.

Click “Review and Create.”

Create data collection endpoint ...

Basics Tags Review + create

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all of your resources. [Learn more](#)

Endpoint details

Endpoint Name *	Custom-Compliance ✓
Subscription * ⓘ	Azure subscription 1 ▼
Resource Group * ⓘ	Intune-Reporting ▼ Create new
Region * ⓘ	Canada Central ▼

[Review + create](#) < Previous Next : Tags >

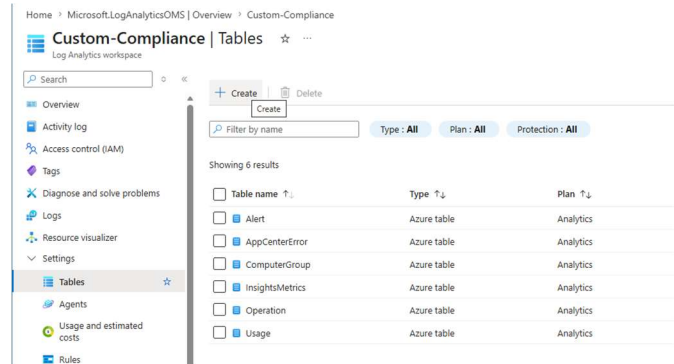
Create the Log Analytics Table and DCR

Let me caution you here. You need to do this once; do it right and fully understand what you are doing at this step. It seems simple from all the Internet documentation, but it really is not and is poorly documented.

If you make a mistake, you cannot delete and re-use the old table and DCR names. The data that underlies them can persist for up to 90 days. The data for the table or DCR does not get deleted and will re-populate in any new one you recreate with the same name. The only other solution is to create one with a different name.

With the log analytics workspace open, expand the “Settings” and select “Tables”.

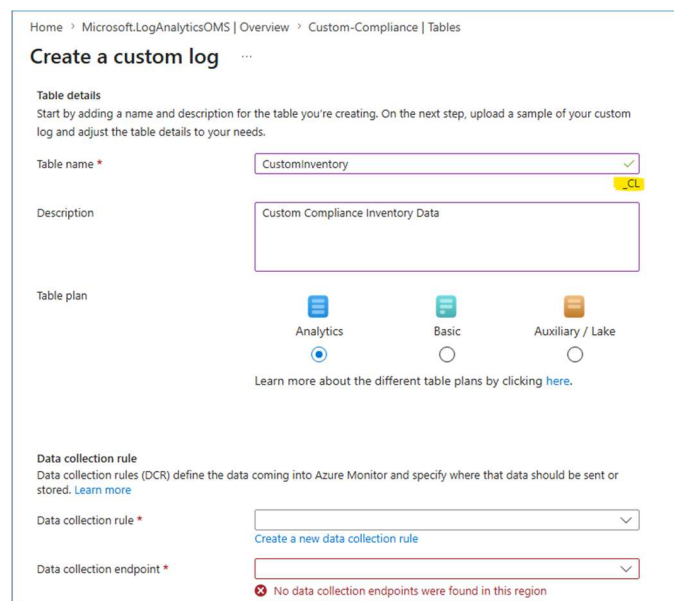
Click the “Create” option.



The table name must be the same as the table you will configure in the script with a few differences. The script uses a stream name which is “Custom-<TableName>_CL”. The “_CL” gets appended to the table name for “Custom Log” in Log Analytics, and the “Custom-” gets added by the DCR to form the data stream name.

Enter a description.

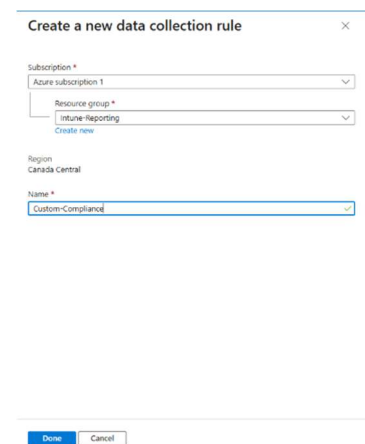
You should not have a DCR yet, so click the “Create a new data collection rule” option.



Choose your subscription and resource group, give the DCR a name.

Click “Done”.

Select the Data Collection Endpoint you created earlier, click “Next”.

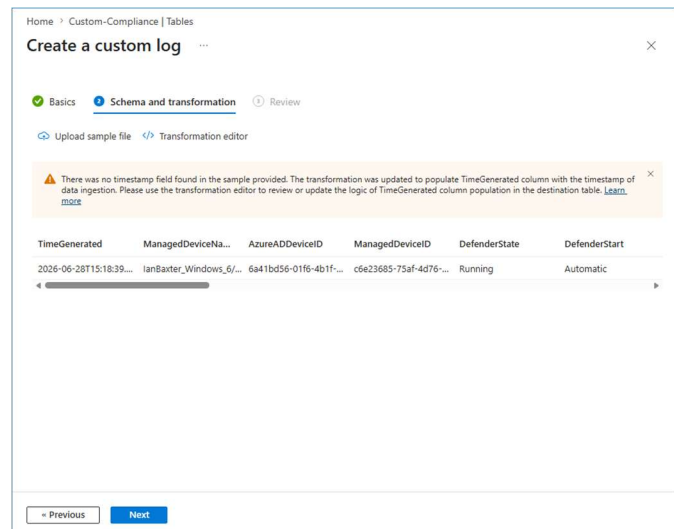
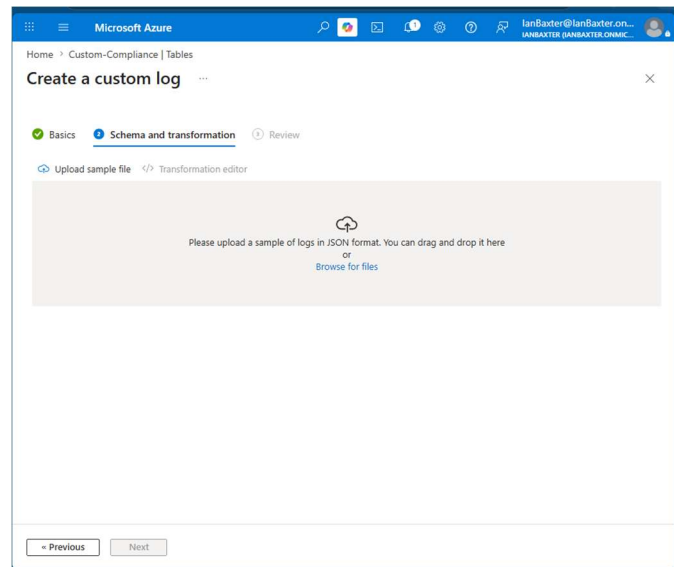


Upload the Json data you saved from the script test.

You should see the warning that a transform has been created for TimeGenerated. If your JSON doesn't contain the mandatory TimeGenerated data, a transform will be automatically created for you.

You will also see all the names of values in the JSON you uploaded.

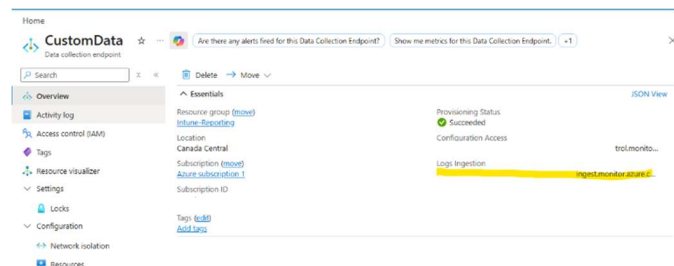
Click “Next” and “Continue”.



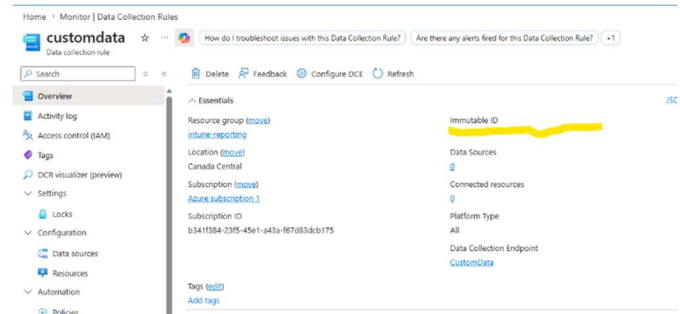
Collect the DCE/DCR details you'll need in code

Record the Logs Ingestion URL from the DCE Overview somewhere so you can use it to configure all the scripts.

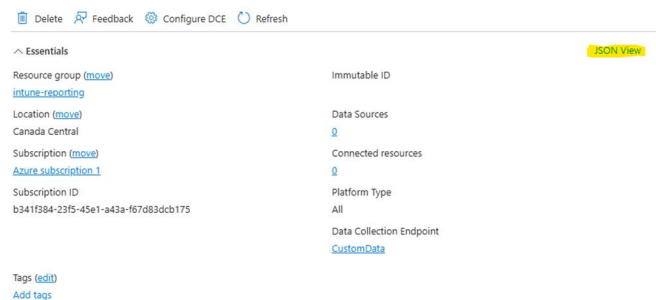
Make a note of the location and Resource Group as well.



Record the DCR Immutable ID from the DCR Overview.



In the DCR Overview, click the “JSON View” link.



Look for this section of the code and record the stream name.

```
9 |         "dataCollectionEndpointId": "/subscriptions/b341f384-23f5-45e1-a43a-f67d83dcb175",
10 |         "streamDeclarations": {
11 |             "Custom-CustomData_Ck": {
12 |                 "columns": [
13 |                     {
14 |                         "name": "TimeGenerated",
15 |                         "type": "datetime"
16 |                     },
17 |                     ...

```

Set up the Azure Function Script

Create the App Service Plan (required for mTLS)

Create the Function App

Enable System Assigned Managed Identity

Enable mTLS (Client Certificate Mode)

Disable HTTP

Create an Azure Vault and store the Root CA PEM

Assign RBAC permissions to the Managed Identity

DCE/DCR

LAW

Key Vault

Azure Function Script (Python)

The easiest way to debug the script is to proceed incrementally and test each block of code from the start to the end. The Function can return a log back to a calling script, so we start with a pre-populated JSON, use the newly installed certificate and then add functionality one step at a time.

Once the function works properly, we can use a semaphore to turn the debug output off and rely on HTTP error codes to report back when an error has occurred.

Logging DCR Issues

Update Inventory Scripts for endpoint URL

Deploy Inventory Scripts